Verification and Validation - Team 4

Objectives

FieldDay's verification and validation process aims to meet essential functional and non-functional requirements. Authenticated users must create complete sports sessions accurately, with approved sessions appearing in discovery within 3 seconds across 50 tests. Users should find and join sessions by geographic radius, sport type, and skill level, achieving zero overbooking from 100 seeded sessions. User profiles must store preferences, skill levels, and details accurately, with updates affecting matching and recommendations within 2 seconds. The recommendation engine will provide at least five ranked session suggestions for active users and relevant suggestions based on preferences for cold-start users. The review system must update aggregate ratings within 1 minute and block invalid submissions like self-ratings, enforcing a 2-day cooldown for users with social ratings below 2.0. Non-functional objectives include maintaining GraphQL API p95 latency below 500 ms under 1,000 concurrent users and ensuring session discovery screens load within 3 seconds on 4G networks. Data must be encrypted with TLS 1.2+ in transit and AES-256 at rest, with 99.5% system uptime and automatic failover within 30 seconds to prevent data loss. First-time users should complete onboarding and join a session in under 5 minutes, achieving an average SUS score of 70 or higher.

Prioritization

Product Capability	Priority	Reason
Creating session	High	Creating a session is an essential feature for users to be able to interact with each other, and forms the key feature of our project. Without the ability to create sessions, all other important features are rendered pointless.
Joining session	High	Joining sessions is a key feature of our application. Without it, the user will have no need of it and as such it must be implemented and fully functional. Should this part of the project fail or function ill, it would cause major disruption to the users
Recommendation engine	High	The ability of our application to provide accurate suggestions to sessions that the

		user may wish to join is another key functionality to what we aim to provide. The application may function without the recommendation engine, purely by sorting using distance and filtering by categories, however it is a major part of the concept that we have, and as such is necessary. If this module malfunctions, users will receive bad recommendations
Interacting with map	Low	A highly useful function that will aid the previous point and make it easier for the users to visually distinguish which events they may be interested in based on location. It is, however, not strictly necessary for the application to function, so it is in effect only an extension to another function.
Location services	High	For the recommendations provided to the user to limit the range of recommended sessions to a realistic and/or desired range, it is necessary that we incorporate a method to see the location of a user and the event. As such, this is a vital feature to the system.
Rating users	Medium	The recommendation engine will make use of the users' opinion of the others they have interacted with previously to make suitable recommendations, as such it is of importance to the performance of the product. It is not, however, a core feature that the application will require, since the system could still provide recommendations based on other factors.
Modifying profile details	High	The recommendation algorithm will rely heavily on the users' preferences listed on their profile, both for the own user's recommendations, as well as that of others. Given the entire system relies on providing groupings that are fitting to the user, this is a very important part to implement for the final product.
Social features, messaging	Low	Another feature that will improve the general user experience, but does not provide any service that other parts of the project rely on, nor does it provide the

		user with an essential service which they would utilize the application for.

Full RVTM

Link to FieldDay RVTM:
https://docs.google.com/spreadsheets/d/1QHR3RqTfouCefUX1OIq7wSHMFg9Z2Mq0PCExWngCn84/edit?usp=sharing

Test Approach

FieldDay will use a testing strategy that verifies both core functionality and the non-functional attributes that are central to the product's success. Any failures in session joining and discovery, recommendation quality, geospatial performance, deployment reliability, trust-related features and non-functional requirements such as system performance, data privacy and security, reliability and usability would have a great impact on both user adoption and system credibility.

Plans for Unit, Integration and System Testing

At the unit testing level, we will verify isolated business rules and internal logic before those components are integrated into the wider system. These tests will be implemented primarily with Jest, and they will focus on modules that contain deterministic application rules rather than external-service behavior. For session creation and scheduling, we will write unit tests for the validation logic that checks required fields such as sport, date/time, location, participant limit, and skill level, and we will include boundary cases such as missing fields, invalid time ordering, and out-of-range participant counts. For user profile and preference logic, we will test validation of selected sports, skill-level values, availability blocks, and profile update rules. For social trust features, we will use boundary-value unit tests to verify rating aggregation, rolling-average calculations, and cooldown threshold logic, including cases such as a user dropping below 2.0, remaining exactly at 2.0, and recovering above the threshold. For authorization and permission logic, we will use mocked user identities and mocked session membership data to verify that only authorized users can read or modify protected data, submit ratings for sessions they attended, or access chat and review actions associated with a joined session. For recommendation support logic, we will unit test the code that builds the Shaped AI request payload to confirm that the outgoing request always contains the correct user attributes, session attributes, and interaction fields before the backend ever sends it to the external recommendation service. These unit tests will be automated and run continuously because they are the fastest way to detect logic regressions when the team changes core business rules.

At the integration testing level, we will verify that FieldDay's major subsystems communicate correctly with one another. These tests will use Jest for integration orchestration, Postman for API-level verification, and a dedicated test database in MongoDB Atlas so that database behavior can be exercised without affecting production-like data. For mobile-to-backend integration, we will test GraphQL queries and mutations for flows such as account profile retrieval, session creation, session discovery, session joining, and post-session rating submission, verifying not only HTTP success but also response structure, returned data fields, and enforcement of business constraints. For backend-to-database integration, we will seed known user, session, rating, and friendship data into the test database and then execute real GraphQL operations against it to verify persistence, roster updates, social rating updates, and read/write correctness. For geospatial functionality, we will load sessions with known coordinates into MongoDB Atlas and verify that discovery queries filtered by radius, sport, and skill level return the correct sessions while excluding those outside the configured range. For third-party integrations, we will test Shaped AI with sandbox or mocked recommendation responses to confirm request formatting, response parsing, error handling, caching behavior, and cold-start fallback behavior; Google Maps integration with known addresses and coordinates to verify geocoding and location resolution; Google OAuth with test identities to verify token validation and session establishment; SheerID with valid, invalid, and timeout scenarios to verify student verification handling; and AWS SNS with test notifications to confirm that the backend can successfully generate and route notification requests. Most of these integration tests will be automated, but we will also perform manual staging checks with real service credentials before milestones to verify that cloud networking, secrets configuration, and actual provider responses match our mocked and sandboxed expectations.

At the system testing level, we will verify complete end-to-end user workflows under realistic conditions using a combination of automated smoke checks and manual walkthroughs on emulators and physical mobile devices. We will group these workflows into the most important product journeys. The first group is the account creation and profile setup flow: using a test Google account, the tester will sign in through Google OAuth, complete the onboarding fields, select at least one sport and skill level, add availability, and verify that the profile persists after logout and login. The second group is the session lifecycle flow: a test user will create a session with a defined sport, time, location, skill level, and participant limit; a second user will discover the same session through filtering and map-based browsing; that second user will join the session; and the team will verify that participant counts, session visibility, and session details remain synchronized across users. The third group is the post-session trust and social flow: after the session is marked complete, participants will submit ratings, the system will update the rated user's aggregate social score, optional friend connections will be created where selected, and low-rating scenarios will be tested to confirm that cooldown restrictions are enforced correctly. The fourth group is the `recommendation and location` flow: seeded users with different sports, schedules, locations, and interaction histories will request recommendations, and the returned

sessions will be checked for geographic relevance, sport relevance, and skill compatibility; the same users will then use the map-based session view to confirm that nearby sessions render correctly and that distance filtering behaves as expected. System testing is where we verify that the product behaves like a real FieldDay user would experience it, so manual execution on real devices will remain essential even when smoke tests are automated.

Manual vs Automated Testing

Our manual versus automated testing split is based on the nature of each task, but it is grounded in the specific tests described above rather than in broad categories alone. Automated tests will include the Jest unit tests for validation logic, permission logic, rating and cooldown calculations, and recommendation request construction; the automated GraphQL and database integration tests for session, profile, recommendation, and rating flows; Postman API collections for repeatable verification of backend operations; and CI-triggered regression suites for the most important workflows. Manual testing will include the complete end-to-end walkthroughs for Google sign-in and onboarding, real-device map and location validation, student verification checks, exploratory testing of social flows, push notification delivery checks through AWS SNS, and reliability checks where human observation is needed to confirm reconnection, usability, or user-facing behavior. In other words, the team will automate what is repeatable and deterministic, and we will keep manual testing for what must be judged in realistic mobile conditions or with direct stakeholder observation.

Plans for Testing Non-Functional Requirements

For non-functional requirements, we will verify performance, security, usability, and reliability as first-class concerns rather than treating them as secondary checks at the end of development. Performance testing will focus on the GraphQL operations that matter most to the app experience, especially session discovery queries, profile retrieval, session join mutations, and recommendation requests. Using k6, we will simulate concurrent traffic against these operations and measure response-time thresholds such as p95 latency, total error rate, and throughput, with special attention to the requirement that the API stay within 300 ms for at least 95% of requests under the target load. We will also measure the full session discovery experience, including backend query time and client rendering time, because map loading and list population are central to the FieldDay user journey.
Security testing will verify Google OAuth authentication behavior, token validation, role-based access controls, restricted access to joined-session resources, and secure handling of sensitive user data. We will use Postman to test authorized and unauthorized requests, confirming that one user cannot access another user’s protected profile, friend data, or session-specific communication without the required permissions, and we will use OWASP ZAP to scan the

staging API for common vulnerabilities such as injection, broken authentication, and misconfigured endpoints.
Usability testing will focus on whether a first-time user can sign in, complete a profile, and join a session quickly without external help, and we will collect task-completion times, error counts, hesitation points, and structured usability feedback from test participants.
Reliability testing will verify health checks, load balancer routing, and recovery behavior in the EC2 and Elastic Load Balancing environment. We will deploy at least two backend instances, simulate an instance failure, observe how quickly ELB routes traffic to healthy instances, and verify that the application continues to serve requests without losing committed data or leaving users in a broken state.

Regression Testing Approach

Our regression testing approach will be continuous throughout the development lifecycle and will be tied directly to the specific workflows that matter most to FieldDay. Every pull request will trigger automated Jest unit tests and automated API-level integration checks in GitHub Actions. These runs will confirm that session validation still works, permission checks still block unauthorized actions, recommendation request formatting has not changed unexpectedly, social rating and cooldown logic still calculate correctly, and core GraphQL operations still return the expected fields and values. In addition, we will maintain a fixed Postman regression collection for the most important backend workflows: profile retrieval, session creation, session discovery, session joining, rating submission, recommendation retrieval, and student verification where enabled. Whenever a major subsystem changes, such as location services, authentication, recommendations, or rating logic, the team will rerun both the automated regression suite and a targeted manual regression pass on the affected user flows. Before milestone reviews and final delivery, we will execute a full regression cycle that includes automated results from Jest, Postman, and k6, followed by manual confirmation of the highest-value mobile workflows on real devices. Regression results will be judged against expected outputs, response schemas, latency thresholds, and previously archived reports so that the team can quickly see whether a new change has broken an established behavior.

Validation Plan

Usability Success Metrics:
Determining the quality of interaction between the user and the project application will constitute a major part of the success of our final product. As such, there are a few categories that are important to consider from the pure interaction point of view. Most of these metrics would be measured in a final period of the project, compared to the more technical testing.

Metric	Measurement Approach	Success Threshold
Visual Smoothness	Query the users whether they notice any visual stutter in the UI while navigating through the application.	Users do not report any sign of stuttering.
Satisfaction Query	Ask a user about how satisfied they are with the process of joining a session and later reviewing it.	Average score above 7.5 out of 10.
Ease of Navigation	Measure the number of users that go to a part of the app, only to go back instead of interacting with it.	Under 20% of users return from a screen without interacting with its contents.

Performance Success Metrics:
These tests will be performed over time as the application and backend are being developed, with the success threshold being the "goal" rather than the initial expectation. These tests will be carried out while a component of the project is being developed, as well as when it is finished, before it is tested after it has been integrated with the rest of the components.

Metric	Measurement Approach	Success Threshold
Input Error Rate	Measure how often a user input fails to register due to communication with the backend.	Fewer than 1 reported error per 30 interactions.
Discovery to Join Time	Measure the amount of time taken from launching the app to successfully joining a session.	Less than five minutes from launching the application.
Geographic Query Response Time	Run a test script to measure the amount of time it takes for the Google Maps SDK to receive an input and provide a valid response.	Throughout a batch of tests, an average response time of ≤ 500ms.
Load Capacity	Check whether the server is able to reliably handle spikes	No recorded lost data during load testing.

	in requests by limiting its available resources and running test scripts.	
Map Render Speed	Use an Android/iOS profiler during a system test with a number of nearby available sessions while navigating through the map.	Rendering thread records ≤ 15ms delay per frame during panning.

Pilot Testing Metrics:
These metrics will also be tested during more final periods of the development, with initial smaller samples before scaling up to larger numbers of users. It will take time to gather the necessary amount of samples to be able to verify whether the application is functioning to our desired expectations, and as such it will likely that we run preliminary tests on isolated parts with smaller user groups, before the testing of the application as a whole is done.

Metric	Measurement Approach	Success Threshold
Session Joining Rate	Measure the percentage of users that join a recommended session versus those that go back to browse further.	At least 20% of users who view a recommended session detail screen join the session.
Post-Session Rating Completion	Measure the percentage of users that complete a session and submit ratings of other users afterwards.	At least 60% of users complete the post-session review.
Likelihood of Recommendation	Query the user after five participated sessions on whether they would recommend FieldDay to someone else.	With a scale of -3 to 3, an average score of 1.5.

Tools, Environments, and Test Data Sets

The FieldDay team will use the following tools as the core verification and validation toolchain:

- Jest for automated unit tests and backend integration tests.

- React Native Testing Library for component-level and screen-level mobile UI testing.
- Postman for GraphQL API collections, request assertions, and repeatable regression checks.
- k6 for load and performance testing of critical API workflows.
- OWASP ZAP for automated security scans of the staging API.
- SSL Labs for verification of TLS and deployed endpoint transport security.
- GitHub for version control, pull request review, issue tracking, and maintaining the history of test-related changes.
- GitHub Actions for CI/CD execution, automated test runs, and archiving of reports.
- Expo and physical iOS/Android devices for manual and device-level validation.

Jest will be configured as the main automated test runner for backend logic and service integration code. It will run the validation, rating, cooldown, authorization, and recommendation-construction tests described in the test approach section, and it will also run integration tests that exercise GraphQL operations against a dedicated test database. For frontend-focused automated checks, React Native Testing Library will be used with Jest to verify that important screens render the correct state, respond to user inputs, and display expected data for flows such as onboarding, session browsing, and rating submission.

Postman will be configured with separate local, staging, and cloud environment variables so that the same API test collections can be executed safely against multiple environments. These collections will include assertions for response status, GraphQL response shape, required fields, authorization behavior, and core business outcomes such as whether a session roster updates correctly or whether an unauthorized request is blocked. The same collections will also be reused as repeatable regression checks whenever a backend feature changes.

k6 will be used as the single load-testing tool for FieldDay. We will configure k6 scripts to simulate increasing numbers of virtual users against the most important GraphQL operations, especially session discovery, profile retrieval, recommendation retrieval, and session-join behavior. These scripts will be configured around the project's performance requirements, including thresholds such as p95 response time under 300 ms, acceptable request-failure rates, and load patterns that reflect real usage spikes around discovery and joining behavior. This makes the performance configuration requirement-driven rather than arbitrary.

OWASP ZAP will be configured to scan the staging API before major reviews and before final delivery. The scan target will be the deployed GraphQL and supporting endpoints in the staging environment, and the output will be reviewed for broken authentication, exposed endpoints, insecure headers, and other common security issues. SSL Labs will be used separately against the EC2/ELB deployment endpoint to verify TLS configuration and confirm that insecure or deprecated transport settings are not exposed.

GitHub will be used as the version control and collaboration platform for all verification and validation assets. Test scripts, Postman collections, k6 load scripts, seeded test-data definitions, and related test documentation will be stored in the repository so changes can be reviewed through pull requests and traced over time. This will allow the team to see who changed a test, why it changed, and which requirement or defect motivated the update.

GitHub Actions will serve as the execution and archiving layer for the automated verification workflow. Pull requests will trigger automated Jest test runs and selected Postman API checks. Scheduled or milestone-based workflows will run k6 performance scripts and preserve the resulting reports as build artifacts. This gives the team one consistent place to review whether a change passed, failed, regressed, or introduced new warnings.

Our testing will be performed across three primary environments. The first is a local development environment, where developers run unit tests, UI component tests, and isolated integration tests while implementing features. The second is a staging/integration environment, backed by a separate MongoDB Atlas database and separate service credentials, where the team validates subsystem interactions and complete workflows without affecting production-like data. The third is a production-like AWS test environment based on EC2 instances behind Elastic Load Balancing, which will be used for reliability testing, failover testing, load testing, and deployment verification. Mobile validation will be conducted with Expo, iOS and Android emulators, and physical phones owned by team members so that permissions, map behavior, push notifications, and platform-specific UI behavior can be observed directly.

Our test data sets will be realistic, anonymized, and representative of how FieldDay is intended to be used. We will prepare seeded datasets containing user profiles with different sports, skill levels, schedules, and geographic locations; sport sessions across multiple sports, dates, times, and radii; friendship relationships; social rating histories; attendance records; and recommendation interaction histories. We will also create targeted edge-case datasets such as full sessions, users whose ratings place them just below or exactly at the cooldown threshold, users with no recommendation history for cold-start testing, invalid .edu verification attempts, duplicate join attempts, and conflicting or sparse location data. For recommendation and filtering validation, the seeded data will deliberately include users with clearly different preferences so the team can verify whether returned sessions align with the correct sport, location, schedule, and skill signals. For usability and UAT activities, any notes, task times, or feedback gathered from participants will be anonymized before being stored in project records.

These tools and datasets will be configured separately for each environment using environment-specific secrets and configuration files so that local, staging, and cloud-based tests remain isolated from one another. External services such as Shaped AI, Google Maps, SheerID, Google OAuth, and AWS SNS will use mocks, sandbox credentials, or controlled staging

integrations depending on the purpose of the test. Database seeding scripts will be used to reset the staging or test database to a known state before regression, load, and end-to-end testing so that repeated runs are comparable and failures can be reproduced accurately.

Finally, test result archiving will be handled through GitHub Actions artifacts, version-controlled test definitions, and shared team documentation. Automated test logs, coverage summaries, Postman run outputs, k6 performance reports, OWASP ZAP scan results, screenshots and screen recordings (for UI testing), and defect records will be preserved so the team can trace what was tested, what failed, how the issue was resolved, and whether the fix regressed later. This archived evidence will support project management and milestone reviews, ensuring that verification and validation results remain organized, reviewable, and tied to the requirements they are intended to satisfy.